2025

Data Engineering Project- Airline on-time performance analysis

CIS 9760- BIG DATA TECHNOLOGIES

EVELYN NYASEMBI

| evelyn.nyasembi@baruchmail.cuny.edu

Milestone 1

Proposal for Airline Data Analysis

Dataset Description
The dataset contains records of airline performance, including details on flight schedules, delays, cancellations, and other operational metrics. This dataset can be leveraged to analyze trends in airline efficiency, punctuality, and factors influencing delays and cancellations.

Data Source
The dataset is available for download at the following location: Airline on-time performance analysis

Dataset Attributes
The dataset includes 29 columns that capture relevant flight details. Attributes are:

1. **Year** – 1987-2008
2. **Month** – 1-12
3. **DayofMonth** – 1-31
4. **DayofWeek** – 1(Monday) – 7(Sunday)
5. **DepTime** – actual depature time (local, hhmm)
6. **CRSDepTime** – scheduled departure time (local, hhmm)
7. **ArrTime** – actual arrival time (local, hhmm)
8. **CRSArrTime** – scheduled arrival time (local,hhmm)
9. **UniqueCarrier** – unique carrier code
10. **Flight Number** – flight number
11. **Tailnum** – plane tail number
12. **ActualElapsedTime** – in minutes
13. **CRSElapsedTime** – in minutes
14. **AirTime** – in minutes
15. **ArrDelay** – arrival delay, in minutes
16. **DepDelay** – departure delay, in minutes
17. **Origin** – origin IATA airport code
18. **Dest** – destination IATA airport code
19. **Distance** – in miles
20. **TaxiIn** – tax in time, in minutes
21. **TaxiOut** – taxi out time in minutes
22. **Cancelled** – was the flight cancelled
23. **CancellationCode** – reason for cancellation (A = carrier, B = weather, C = NAS, D = security)
24. **Diverted** – 1 = yes, 0 = no
25. **CarrierDelay** – in minutes
26. **WeatherDelay** – in minutes
27. **NasDelay** – in minutes
28. **SecurityDelay** – in minutes
29. **LateAircraftDelay** – in minutes

Prediction Objective:

The goal of this analysis is to predict flight delays using machine learning techniques. Based on the nature of the data and the objectives, **logistic regression** will be used to classify flights as "On Time" or "Delayed" based on whether the delay exceeds a certain percentage (X%) of the total trip time. This method is suitable for categorizing flights effectively while allowing for different delay thresholds (e.g., 10%, 25%, 50%).

Machine Learning Approach

By implement logistic regression to predict whether a delay exceeds a given percentage of total trip time. Additionally, we will experiment with different threshold values (10%, 25%, and 50%) to assess how they impact classification accuracy. Other models such as decision trees or random forests may also be explored for improved performance.

In conclusion, this analysis will provide valuable insights into flight delays, helping airlines and passengers make more informed travel decisions. By identifying key contributing factors to develop better strategies to minimize disruptions in air travel.

Milestone 2

Summary

The data acquisition and storage process involved collecting, storing, and organizing data for further processing. The first step was fetching the dataset from **Kaggle** using a shell script with the curl command. The dataset archive (airline-data.zip) was downloaded directly from the Kaggle repository. After downloading, the dataset was extracted using the unzip command. The extracted files included airline.csv.shuffle and carriers.csv, which were required for analysis.

A **Google Cloud Storage (GCS) bucket** named my-bigdata-project-en was created to store the dataset. The bucket was configured with **standard storage class** and **uniform bucket-level access** to manage permissions and access. Authentication was completed using the gcloud auth login command, allowing interaction with the storage bucket. The extracted dataset files were uploaded to the /landing folder in the GCS bucket using the gcloud storage cp command. A verification step was performed to confirm that the files were successfully transferred to the GCS bucket. The contents of the bucket were listed using gcloud commands.

Additional empty folders were created in the GCS bucket for data processing. These folders included **cleaned**, **trusted**, **code**, and **models**. Empty placeholder files were uploaded to these folders using the gcloud storage cp /dev/null command. These steps established a structured process for data acquisition and storage, ensuring the availability of raw data for further analysis. The structured storage allowed for efficient data processing and integration into subsequent workflows.

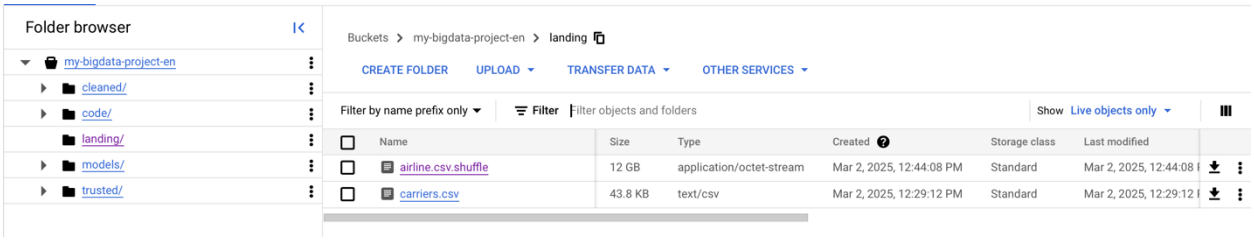
Appendix A Data Acquisition Script

Bucket name:my-bigdata-project-en

1. Fetch the archive file from Kaggle

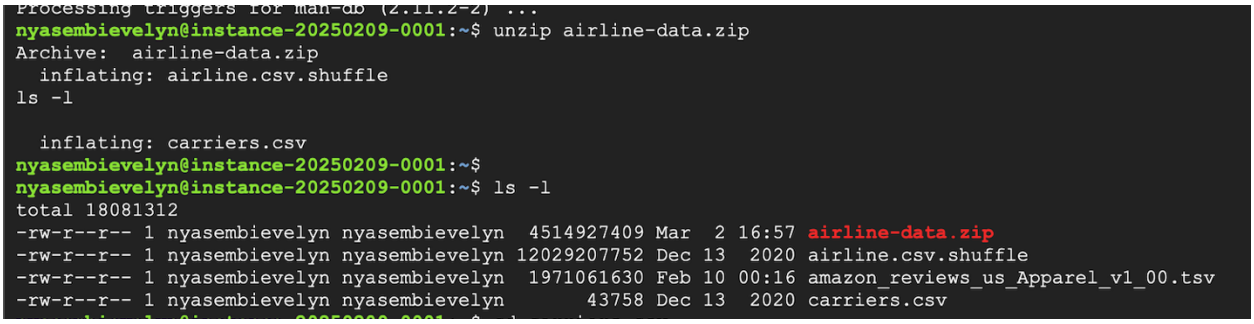
```
#!/bin/bash

curl -L -o airline-data.zip
https://www.kaggle.com/api/v1/datasets/download/bulter22/airline-data
```



2. Unzip the archive file

```
unzip airline-data.zip
```



3. Create a bucket on Google Cloud Storage

```
gcloud storage buckets create gs://my-bigdata-project-en --
project= evelyn-nyasembi-project-1 --default-storage-
class=STANDARD --location=us-central1 --uniform-bucket-level-
access
```

4. Using gcloud auth login to authenticate

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage buckets create gs://my-bigdata-project-en  --project=evelyn-nyasembi-project-1 --default-storage-class=STANDARD --location=us-central1 --uniform-bucket-level-access
Creating gs://my-bigdata-project-en/...
ERROR: (gcloud.storage.buckets.create) HTTPError 403: Access denied. This command is authenticated as 368204563733-compute@developer.gserviceaccount.com which is the active account specified by the [core/account] property.
nyasembievelyn@instance-20250209-0001:~$ gcloud auth login

You are running on a Google Compute Engine virtual machine.
It is recommended that you use service accounts for authentication.

You can run:

  $ gcloud config set account `ACCOUNT`

to switch accounts if necessary.

Your credentials may be visible to others with access to this
virtual machine. Are you sure you want to authenticate with
your personal account?

Do you want to continue (Y/n)?  Y

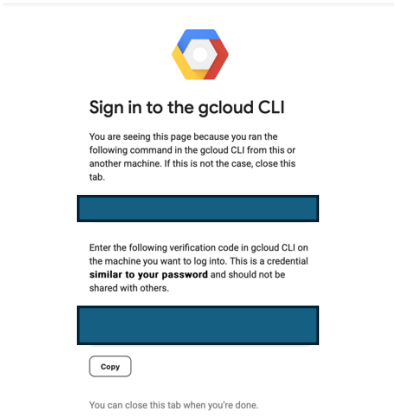
Go to the following link in your browser, and complete the sign-in prompts:

    https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=32555940559.apps.googleusercontent.com&redirect_uri=https%3A%2F%2Fsdk.cloud.google.com%2Fauthcode.html&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-platform+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.login+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcompute+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Faccounts.reauth&state=iZNVOUHL6Fvs7jsDV6vqQybUuEWpyc&prompt=consent&token_usage=remote&access_type=offline&code_challenge=pLuRRW03FLflQFR8ewb2hPZ5X3HyBUcmBptP2j4Xa80&code_challenge_method=S256

Once finished, enter the verification code provided in your browser: 4/0AQSTgQHyf6Rl1M2lxCKH3oZ_fiA9N2CDR69ZMZuxQ_wzyLIR4u41Y85IDUvQgMpfp14H3w

You are now logged in as [nyasembievelyn@gmail.com].
Your current project is [evelyn-nyasembi-project-1].  You can change this setting by running:
  $ gcloud config set project PROJECT_ID


To take a quick anonymous survey, run:
  $ gcloud survey
```



5. Copy the files to GCS bucket in the /landing folder

gcloud storage cp airline.csv.shuffle gs://my-bigdata-project-en/landing/

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage cp airline.csv.shuffle gs://my-bigdata-project-en/landing/
WARNING: Parallel composite upload was turned ON to get the best performance on
uploading large objects. If you would like to opt-out and instead
perform a normal upload, run:
`gcloud config set storage/parallel_composite_upload_enabled False`
If you would like to disable this warning, run:
`gcloud config set storage/parallel_composite_upload_enabled True`
Note that with parallel composite uploads, your object might be
uploaded as a composite object
(https://cloud.google.com/storage/docs/composite-objects), which means
that any user who downloads your object will need to use crc32c
checksums to verify data integrity. gcloud storage is capable of
computing crc32c checksums, but this might pose a problem for other
clients.

Copying file://airline.csv.shuffle to gs://my-bigdata-project-en/landing/airline.csv.shuffle
  Completed files 32/1 | 11.2GiB/11.2GiB | 155.5MiB/s

Average throughput: 149.1MiB/s
```

gcloud storage cp carriers.csv gs://my-bigdata-project-en/landing/

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage cp carriers.csv gs://my-bigdata-project-en/landing/
Copying file://carriers.csv to gs://my-bigdata-project-en/landing/carriers.csv
  Completed files 1/1 | 42.7kiB/42.7kiB
```

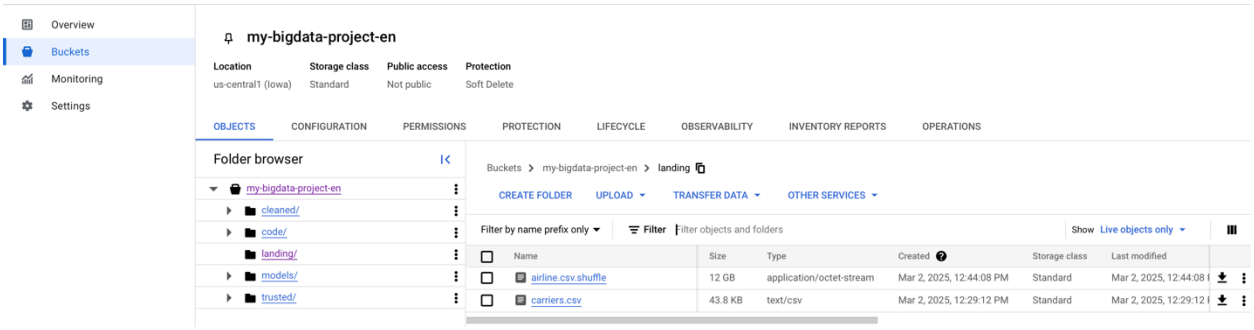
6. List files in gcloud storage

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage ls -l gs://my-bigdata-project-en/landing
12029207752  2025-03-02T17:44:08Z  gs://my-bigdata-project-en/landing/airline.csv.shuffle
     43758  2025-03-02T17:29:12Z  gs://my-bigdata-project-en/landing/carriers.csv
TOTAL: 2 objects, 12029251510 bytes (11.20GiB)
```

7. Create empty folders

```
gcloud storage cp /dev/null gs://my-bigdata-project-en/cleaned
gcloud storage cp /dev/null gs:// my-bigdata-project-en/trusted
gcloud storage cp /dev/null gs:// my-bigdata-project-en/code
gcloud storage cp /dev/null gs:// my-bigdata-project-en/models
```

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage ls -l gs://my-bigdata-project-en
                                 gs://my-bigdata-project-en/cleaned/
                                 gs://my-bigdata-project-en/code/
                                 gs://my-bigdata-project-en/landing/
                                 gs://my-bigdata-project-en/models/
                                 gs://my-bigdata-project-en/trusted/
```



Milestone 3

Main

The airline dataset includes a lot of useful info like delays, flight times, and distances, which makes it solid for analysis. One thing I know for sure is going to be a challenge is working with the time-related columns — especially the ones in HHMM format. When I tried converting them to timestamps, it increased the number of nulls by a lot, which could mess with any time-based features I try to create. For feature engineering, I'll need to be careful with how I handle those conversions, maybe look into rebuilding timestamps using date and time fields together. Besides

that, I'll also need to deal with missing delay values and figure out which features actually help predict delays, like time of day, distance, day of week, or even specific carriers.

Appendix B Source code

1. Using a 5 cluster dataproc node

```
gcloud dataproc clusters create cluster-a8fe --enable-component-gateway --region us-central1 --master-machine-type n2-standard-4 --master-boot-disk-type pd-balanced --master-boot-disk-size 100 --num-workers 2 --worker-machine-type n2-standard-4 --worker-boot-disk-type pd-balanced --worker-boot-disk-size 100 --image-version 2.2-debian12 --optional-components JUPYTER --max-idle 3600s --project ProjectID
```

2. Using PySpark, MatplotLib and Seaborn
   a. Getting data

```
[6]: # Set the logging level to only show errors
     sc.setLogLevel("ERROR")

[7]: # Set up the path to a file
     bucket = 'gs://my-bigdata-project-en'
     landing_folder = f"{bucket}/landing/"
     cleaned_folder = f"{bucket}/cleaned/"

[9]: # Create a Spark Dataframe from some Amazon reviews data
     sdf = spark.read.csv(landing_folder, sep=',', header=True, inferSchema=True)
```

   b. Schema

```
# Check the schema
sdf.printSchema()
```

```
root
 |-- ActualElapsedTime: string (nullable = true)
 |-- AirTime: string (nullable = true)
 |-- ArrDelay: string (nullable = true)
 |-- ArrTime: string (nullable = true)
 |-- CRSArrTime: integer (nullable = true)
 |-- CRSDepTime: integer (nullable = true)
 |-- CRSElapsedTime: string (nullable = true)
 |-- CancellationCode: string (nullable = true)
 |-- Cancelled: integer (nullable = true)
 |-- CarrierDelay: string (nullable = true)
 |-- DayOfWeek: integer (nullable = true)
 |-- DayofMonth: integer (nullable = true)
 |-- DepDelay: string (nullable = true)
 |-- DepTime: string (nullable = true)
 |-- Dest: string (nullable = true)
 |-- Distance: string (nullable = true)
 |-- Diverted: integer (nullable = true)
 |-- FlightNum: integer (nullable = true)
 |-- LateAircraftDelay: string (nullable = true)
 |-- Month: integer (nullable = true)
 |-- NASDelay: string (nullable = true)
 |-- Origin: string (nullable = true)
 |-- SecurityDelay: string (nullable = true)
 |-- TailNum: string (nullable = true)
 |-- TaxiIn: string (nullable = true)
 |-- TaxiOut: string (nullable = true)
 |-- UniqueCarrier: string (nullable = true)
 |-- WeatherDelay: string (nullable = true)
 |-- Year: integer (nullable = true)
```

   c. Number of records

```
# Display the number of records
record_count = sdf.count()
print(f'Total number of records: {record_count}')
```

```
[Stage 2:=============================================================>(89 + 1) / 90]
Total number of records: 123536460
```

d. List of variables

```
# Display the list of columns
columns = sdf.columns
print(f'List of Variables: {columns}')
```

```
List of Variables: ['ActualElapsedTime', 'AirTime', 'ArrDelay', 'ArrTime', 'CRSArrTime', 'CRSDepTime', 'CRSElapsedTime', 'CancellationCode', 'Cancelled', 'Car
rierDelay', 'DayOfWeek', 'DayofMonth', 'DepDelay', 'DepTime', 'Dest', 'Distance', 'Diverted', 'FlightNum', 'LateAircraftDelay', 'Month', 'NASDelay', 'Origin',
'SecurityDelay', 'TailNum', 'TaxiIn', 'TaxiOut', 'UniqueCarrier', 'WeatherDelay', 'Year']
```

e. Count of Missing values

```
# Count missing values per column
na_counts = sdf.select(
    [(F.count(F.when(F.col(c).isNull() | (F.col(c) == 'NA'), c)).alias(c)) for c in sdf.columns]
)

# Display the result
na_counts.show()
```

```
[Stage 23:=============================================================>(89 + 1) / 90]
+----------------+--------+--------+--------+----------+----------+----------------+----------------+---------+------------+---------+----------+--------+----------+--------+------
--+----+--------+--------+--------+----------+----+--------+------+-------------+--------+--------+------+----------+----------+----+
|ActualElapsedTime| AirTime|ArrDelay|ArrTime|CRSArrTime|CRSDepTime|CRSElapsedTime|CancellationCode|Cancelled|CarrierDelay|DayOfWeek|DayofMonth|DepDelay|DepTim
e|Dest|Distance|Diverted|FlightNum|LateAircraftDelay|Month|NASDelay|Origin|SecurityDelay| TailNum|  TaxiIn| TaxiOut|UniqueCarrier|WeatherDelay|Year|
+----------------+--------+--------+-------+----------+----------+----------------+----------------+---------+------------+---------+----------+--------+----------+--------+------
--+----+--------+--------+--------+----------+----+--------+------+-------------+--------+--------+------+----------+----------+----+
|          2587530|39266398| 2589020|2585969|      1491|      1491|           27725|       122801754|     1491|    89330924|     1491|      1491| 2303627|230362
7|1491|  203491|    1491|     1491|         89330924| 1491|89330924|  1491|     89330924|37386911|37398786|37384195|         1491|    89330924|1491|
+----------------+--------+--------+-------+----------+----------+----------------+----------------+---------+------------+---------+----------+--------+----------+--------+------
--+----+--------+--------+--------+----------+----+--------+------+-------------+--------+--------+------+----------+----------+----+
```

f. Min/Max Dates

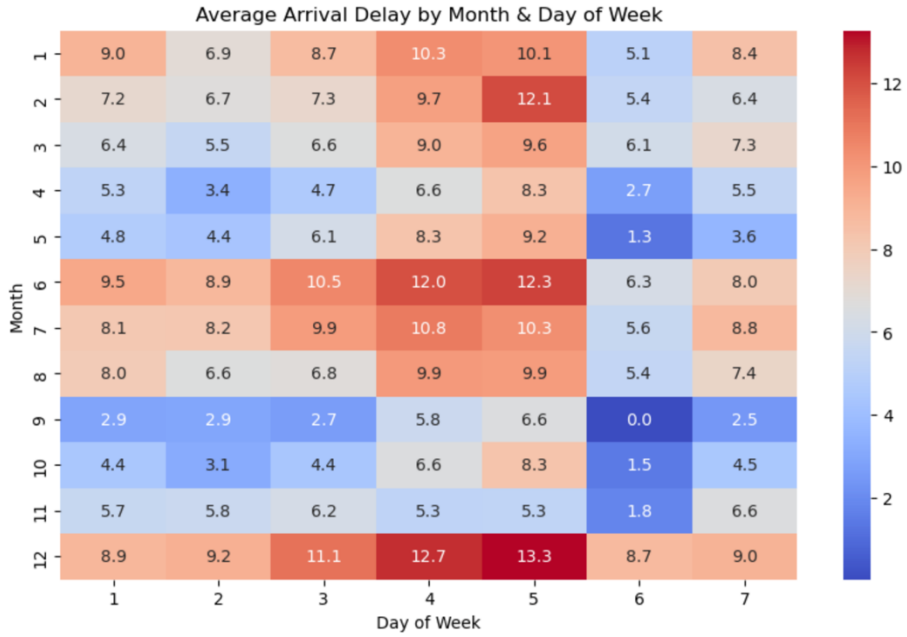
```
# min and max dates.
sdf.select("DayOfWeek", "DayofMonth", "Month","Year").summary("count", "min",
"max").show()
```

```
[Stage 5:=============================================================> (88 + 2) / 90]
+-------+---------+----------+---------+---------+
|summary|DayOfWeek|DayofMonth|    Month|     Year|
+-------+---------+----------+---------+---------+
|  count|123534969| 123534969|123534969|123534969|
|    min|        1|         1|        1|     1987|
|    max|        7|        31|       12|     2008|
+-------+---------+----------+---------+---------+
```

g. graphs / charts to show the distribution of data

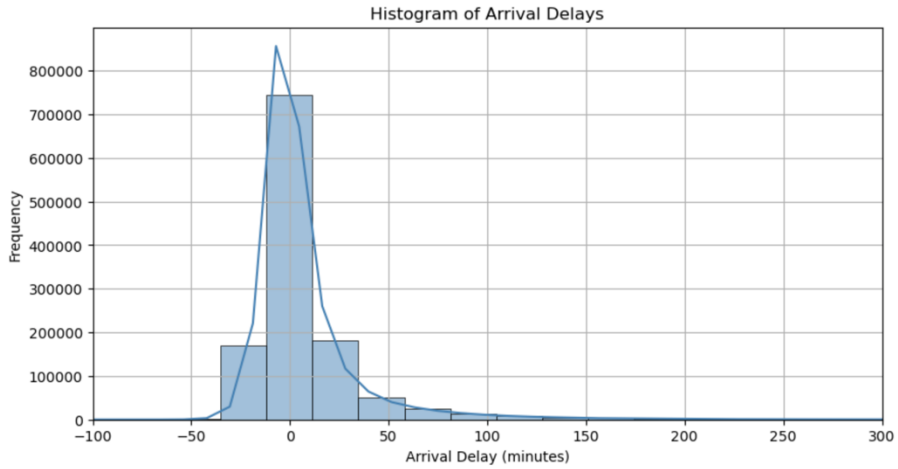
```
delay_heatmap = sdf.groupBy("Month", "DayOfWeek") \
                   .avg("ArrDelay") \
                   .toPandas().pivot(index="Month", columns="DayOfWeek", values="avg(ArrDelay)")

plt.figure(figsize=(10, 6))
sns.heatmap(delay_heatmap, annot=True, fmt=".1f", cmap="coolwarm")
plt.title("Average Arrival Delay by Month & Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Month")
plt.show()
```




```
# Sample and convert to pandas to keep it fast and memory-safe
pandas_df = sdf.select("ArrDelay").dropna().sample(False, 0.01).toPandas()

# Plot histogram
plt.figure(figsize=(10, 5))
sns.histplot(pandas_df["ArrDelay"], bins=100, kde=True, color="steelblue")
plt.title("Histogram of Arrival Delays")
plt.xlabel("Arrival Delay (minutes)")
plt.ylabel("Frequency")
plt.xlim(-100, 300)  # Optional: zoom into the most common delay range
plt.grid(True)
plt.show()
```

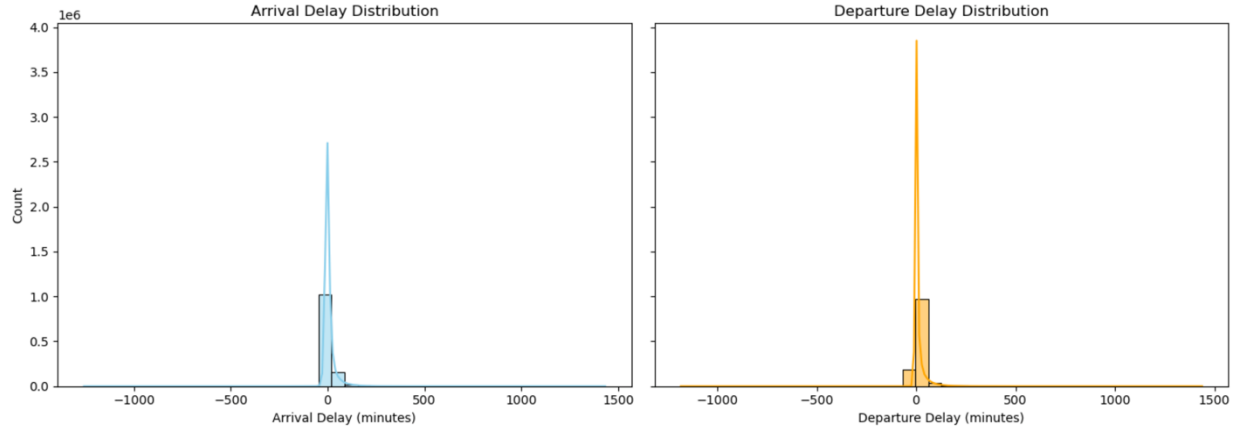


```
fig, axs = plt.subplots(1, 2, figsize=(14, 5), sharey=True)

sns.histplot(pandas_df["ArrDelay"], kde=True, bins=40, color="skyblue", ax=axs[0])
axs[0].set_title("Arrival Delay Distribution")
axs[0].set_xlabel("Arrival Delay (minutes)")

sns.histplot(pandas_df["DepDelay"], kde=True, bins=40, color="orange", ax=axs[1])
axs[1].set_title("Departure Delay Distribution")
axs[1].set_xlabel("Departure Delay (minutes)")

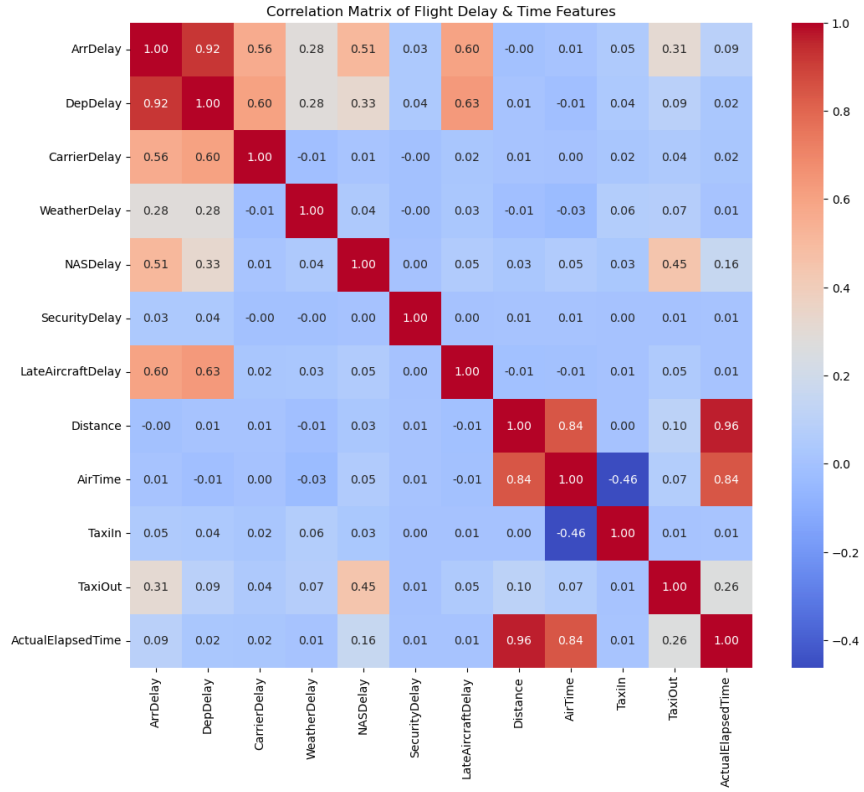
plt.tight_layout()
plt.show()
```



```
# Correlation Heatmap (for numeric relationships) Strong positive correlation (close to 1) → variables rise together
#Strong negative correlation (close to −1) → one rises, the other falls
#Near 0 → weak or no linear relationship
cols = [
    "ArrDelay", "DepDelay", "CarrierDelay", "WeatherDelay",
    "NASDelay", "SecurityDelay", "LateAircraftDelay",
    "Distance", "AirTime", "TaxiIn", "TaxiOut", "ActualElapsedTime"
]

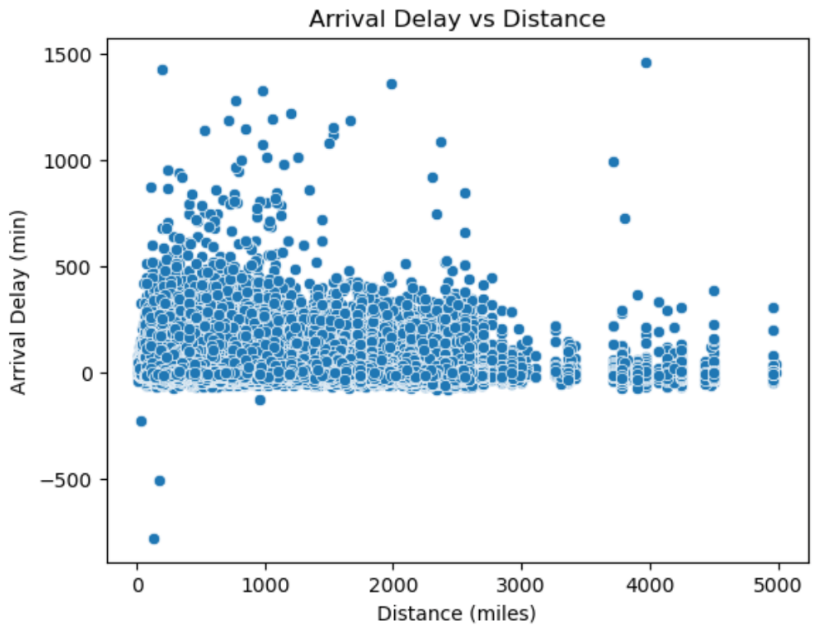
# Sample and convert to pandas
numeric_df = sdf.select(cols).dropna().sample(False, 0.01).toPandas()

plt.figure(figsize=(12, 10))
sns.heatmap(numeric_df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix of Flight Delay & Time Features")
plt.show()
```



```
#Scatter plots (for checking relationships between two specific columns)
# Delay vs Distance
pair_df = sdf.select("ArrDelay", "Distance").dropna().sample(False, 0.01).toPandas()

sns.scatterplot(x="Distance", y="ArrDelay", data=pair_df)
plt.title("Arrival Delay vs Distance")
plt.xlabel("Distance (miles)")
plt.ylabel("Arrival Delay (min)")
plt.show()
```



Appendix C Cleaned Data code

a. removing records with missing data

```
[10]: # Check to see if some of the columns have NULL values
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in ["ActualElapsedTime", "AirTime", "AirTime", "CRSArrTime", "CRSDepTime","ArrDelay","Dep
```

```
[Stage 8:=====================================================> (88 + 2) / 90]
+-----------------+-------+-------+----------+----------+--------+--------+--------+------+-------+-----+----+---------+
|ActualElapsedTime|AirTime|AirTime|CRSArrTime|CRSDepTime|ArrDelay|DepDelay|Distance|TaxiIn|TaxiOut|Month|Year|DayOfWeek|
+-----------------+-------+-------+----------+----------+--------+--------+--------+------+-------+-----+----+---------+
|                0|      0|      0|      1491|      1491|    1491|    1491|    1491|  1491|   1491| 1491|1491|     1491|
+-----------------+-------+-------+----------+----------+--------+--------+--------+------+-------+-----+----+---------+
```

h. Missing values were less than 10%

```
# Drop some of the records where the certain columns are empty (null or na)
sdf = sdf.na.drop(subset=["ActualElapsedTime", "AirTime", "AirTime", "CRSArrTime", "CRSDepTime","ArrDelay","DepDelay","Distance","TaxiIn","TaxiOut","Month","Ye
```

```
# Get the new number of records after dropping
print("After dropping records")
sdf.count()
```

```
After dropping records

123534969
```

```
erify if nan dropped
.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in ["ActualElapsedTime", "AirTime", "AirTime", "CRSArrTime", "CRSDepTime","ArrDelay","DepDel
```

```
[Stage 14:=============================================================> (88 + 2) / 90]
+-----------------+-------+-------+----------+----------+--------+--------+--------+------+-------+-----+----+---------+
|ActualElapsedTime|AirTime|AirTime|CRSArrTime|CRSDepTime|ArrDelay|DepDelay|Distance|TaxiIn|TaxiOut|Month|Year|DayOfWeek|
+-----------------+-------+-------+----------+----------+--------+--------+--------+------+-------+-----+----+---------+
|                0|      0|      0|         0|         0|       0|       0|       0|     0|      0|    0|   0|        0|
+-----------------+-------+-------+----------+----------+--------+--------+--------+------+-------+-----+----+---------+
```

i. Dropping irrelevant columns (doesn't change number of records)

```
sdf = sdf.drop("FlightNum", "TailNum", "Year")
```

j. rename columns and removed spaces

```
# Saving the final dataset as a Parquet file
# Actually this will save as a folder with a file for each of the 90 partitions.
cleaned_filename = f"{cleaned_folder}/airline-data.parquet"
sdf.write.parquet(cleaned_filename, mode='overwrite')
```

```
25/03/30 00:13:27 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
```

